

Sistema de gestión de rutinas de entrenamiento personalizadas

Contexto del problema

En un centro de entrenamiento deportivo, los instructores deben diseñar rutinas personalizadas para sus clientes según sus objetivos físicos. Estas rutinas pueden incluir distintos tipos de ejercicios y cada ejercicio debe seleccionarse en función del nivel de exigencia que se desea trabajar.

Los niveles de intensidad que maneja el centro son los siguientes:

- Básico
- Intermedio
- Avanzado
- Alto rendimiento

Cada rutina está asociada a un cliente en particular y, dependiendo de su objetivo físico, podrá tener una duración total distinta del resto.

Suponga que cada rutina está compuesta por un conjunto indeterminado de ejercicios de dos tipos:

- Ejercicios cardiovasculares
- Ejercicios de fuerza

Cada ejercicio responde a un nivel de intensidad, además de poseer un tiempo estimado para ser realizado por el cliente. Los ejercicios no pueden repetirse en rutinas de dos semanas consecutivas para un mismo cliente y deben tener una descripción clara de su ejecución esperada.

Cada ejercicio debe contemplar, al menos, la siguiente información:

- Código identificador
- Nombre del ejercicio
- Tipo de ejercicio
- Nivel de intensidad
- Tiempo estimado en minutos
- Descripción de ejecución
- Semana en la que fue usado por última vez

Instrucciones

Desarrolle un sistema con interfaz gráfica de usuario que administre la aplicación de rutinas de entrenamiento personalizadas, bajo las indicaciones formuladas anteriormente.

Las funcionalidades del sistema deben contemplar lo siguiente:

1. Carga de ejercicios desde archivo ó desde una base de datos(BD)

Los datos de los ejercicios que compondrán la rutina deben cargarse al iniciar el programa. Esta carga debe realizarse mediante la selección de un archivo o BD ubicado en el espacio de almacenamiento donde se esté ejecutando el programa.

Una vez cargados correctamente los ejercicios en un arreglo, el sistema debe desplegar en la interfaz gráfica la siguiente información: cantidad total de ejercicios cargados, tiempo estimado total disponible, cantidad de ejercicios por tipo y cantidad de ejercicios por nivel de intensidad.

Además, se debe mostrar un botón que permita iniciar la generación de la rutina de entrenamiento.

2. Generación de rutina

Una vez cargados los ejercicios, el sistema debe permitir generar una rutina de entrenamiento solicitando:

- Cantidad de ejercicios deseados por cada tipo (cardiovascular y fuerza).
- Nivel de intensidad requerido por cada ejercicio.

El sistema deberá construir la rutina de entrenamiento considerando:

- Ejercicios disponibles.
- Nivel de intensidad solicitado.
- Restricción de no repetición en semanas consecutivas.

3. Revisión de la rutina

El sistema debe ofrecer la posibilidad de revisar la rutina creada, para esto, el sistema debe mostrar un ejercicio a la vez, incluyendo:

- Nombre del ejercicio.
- Tipo de ejercicio.
- Nivel de intensidad.
- Tiempo estimado.
- Descripción de ejecución.

El sistema debe mantener el estado de navegación entre ejercicios, es decir, debe permitir las siguientes acciones:

- Avanzar al siguiente ejercicio
- Volver al ejercicio anterior

Si el usuario se encuentra en el primer ejercicio, la opción "Volver" debe estar deshabilitada.

Si el usuario se encuentra en el último ejercicio, el botón "Siguiente" debe cambiar a "Resumen de la rutina".

Al hacer clic sobre el botón "Resumen de la rutina", el sistema debe mostrar:

- Cantidad de ejercicios por cada tipo (cardiovascular y fuerza).
- Cantidad de ejercicios por nivel de intensidad.
- Tiempo total estimado de la rutina.

Requerimientos

Lenguaje de programación e interfaz gráfica de usuario: el sistema debe desarrollarse utilizando la biblioteca Swing de Java y debe considerar al menos las siguientes ventanas:

- Ventana principal
- Ventana de carga de archivo
- Ventana de generación de rutina
- Ventana de revisión de rutina
- Ventana de resumen de rutina

Persistencia de información: el sistema debe cargar los ejercicios desde archivo y mantenerlos en memoria durante la ejecución del programa. Se debe validar: formato incorrecto, información incompleta y archivo inexistente. El manejo de errores debe realizarse mediante excepciones. El formato del archivo queda a criterio de los estudiantes.

Arquitectura del sistema: la aplicación debe estar modularizada en dos paquetes: backend y frontend, que representan respectivamente la lógica de la aplicación y la gestión de la interfaz gráfica. La comunicación entre estos paquetes debe ser mediante notificación-suscripción, en donde eventos de la interfaz generen mensajes al backend, y luego las actualizaciones de este último sean notificadas de manera asincrónica al frontend.

Documentación: se debe incluir un archivo *leame* con todos los alcances y supuestos incorporados, como también instrucciones para la ejecución del sistema, incluyendo la organización de los datos en el archivo de ejercicios y la forma en que debe ser cargado.

Buenas prácticas: el sistema debe mantener modularidad (separación de archivos), aplicar los fundamentos de la programación orientada a objetos e implementar la programación orientada a eventos.

Informe

Desarrolle un informe que describa las decisiones de diseño. El informe debe contener:

- Portada: identificación de la universidad, facultad, carrera y asignatura. Título del trabajo o Tarea. Nombre y RUT de cada integrante del equipo, NRC, ciudad y fecha de entrega del informe.
- Introducción: motivación, objetivo del trabajo y estructura del documento.

- Descripción de la solución: explicación del modelo propuesto, clases diseñadas, relaciones entre ellas y justificación de decisiones. Además, elaborar el diagrama de clases de la solución propuesta.
- Conclusión: reflexión sobre el trabajo realizado, cumplimiento de objetivos y aprendizajes obtenidos.

El informe y el software deben ser entregados en la fecha indicada por el profesor. El equipo de trabajo debe estar conformado por un mínimo de tres estudiantes y un máximo de cinco estudiantes.